COMP 2123

Lectures

L2 - Lists

Abstract Data Types (ADT)

- Type defined in terms of its data items and associated operations, not its implementation
- Abstract Data Type (ADT) is a conceptual model. They define what a data structure does without dictating how it does it. By separating the logical properties from the physical implementation, ADTs allow for cleaner, more maintainable code.

Benefits of ADT Approach

- Code is easier to understand if different issues are separated into different places
- Client can be considered at a higher more abstract level
- Many different systems can use the same library, so only code tricky manipulations once, rather than in every client system
- There can be choices of implementations with different performance tradeoffs, and the client doesn't need to be rewritten extensively to change which implementation it uses

Challenges of ADT Approach

- Specify how to deal with boundary cases
  - Negative numbers
  - Double calls (e.g. reserve a seat thats already been reserved)
  - 0
- We can "adapt" existing ADT structure with our own inputs as well

Index-Based Lists (List ADT)

- An index-based list (usually) supports the following operations:
  - size()
  - isEmpty()
  - get(i)
  - set(i, e) - returns value at i before e replaces
  - add(i, e) - shifts everything >= i up
  - remove(i) - shifts everything >= i down, returns value e

Array-Based List

- An option for implementing the list ADT is to use an array A, where A[i] stores (a reference to) the element with index i.
- If array has size N then we can represent lists of size n<=N
  How is an arr

Data Structure

- A data structure is a concrete representation of data, and this is from the point of view of an implementer, not a user

L3

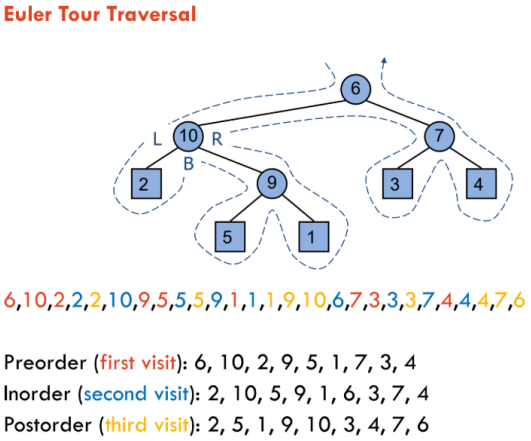
Trees

Generic methods:

- integer size()
- boolean isEmpty()
- Iterator iterator()
- Iterable positions()
- Access methods:
- Position root()
- Position parent(p)
- Iterable children(p)
- Integer numChildren(p)

Tree Traversal

Euler Tour Traversal(Visit all nodes)



Binary Trees

Trees with maximum 2 children

L4

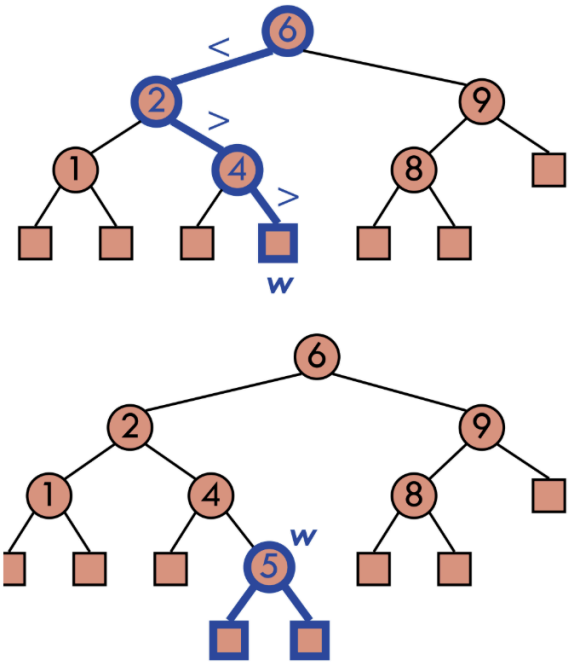
Binary Search Trees (BST)

A binary search tree is a binary tree storing keys (or key-value pairs) satisfying the following BST property:

> For any node v in the tree and any node u in the left subtree of v and any node w in the right subtree of v, key(u) < key(v) < key(w)

Note that an inorder traversal of a binary search tree visits the keys in increasing order.

Insertion
always happens in leaf node



Deletion

1. has 2 external child: delete directly
2. has 1 external child: replace the deleted node with child
3. has 2 internal children: replace the deleted node with inorder successor, i.e. the left-most node in right subtree

BST Searching

Runs in O(h) time, where h is the height of the tree. Best case $h = logn$, worst case $h = n - 1$

Range Queries

Are just restricted inorder traversal searches

Rank Balanced Trees

Aim to maintain a balanced BST to gaurantee average of O(logn) search.

AVL Tree

Self balancing tree. Behaves exactly like a BST but every node has a a requirement:

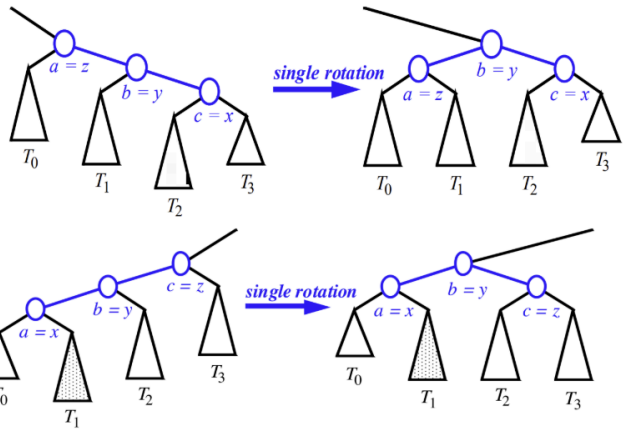
$$|\text{height of left subtree} - \text{height of right subtree}| \leq 1$$

If this is violated, the tree must fix this using rotations - trinode restructuring.

Single Rotation

Trinode Restructuring (when done by Single Rotation)

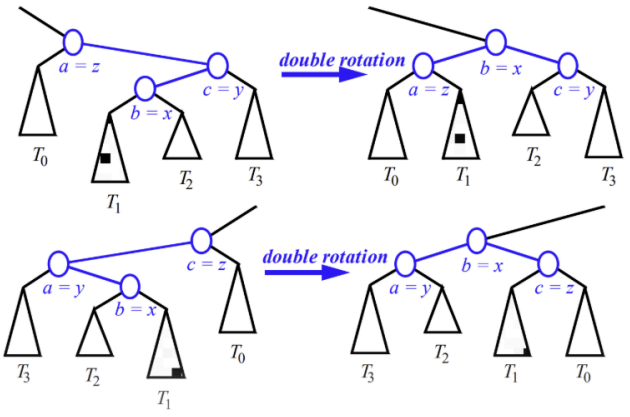
Single Rotations:



Double Rotation

Trinode Restructuring (when done by Double Rotation)

Double rotations:



Suppose we have an AVL tree storing n items then

- The data structure uses O(n) space
- Height of the tree O(log n)
- Searching takes O(log n) time
- Insertion takes O(log n) time
- Removal takes O(log n) time

L5

Priority Queue

Special type of ADT map to store a collection of key-value items where we can only remove smallest key. Does not support searching for an element. Can also have a max/min version.

Operations

- **insert(k, v)**: insert item with key k and value v
- **remove_min()**: remove and return the item with smallest key
- **min()**: return item with smallest key
- **size()**: return how many items are stored
- **is_empty()**: test if queue is empty

Sequence Based Priority Queue

Unsorted list Implementation

- **insert** in O(1) time since we can insert the item at the beginning or end of the sequence
- **remove_min** and **min** in O(n) time since we have to traverse the entire list to find the smallest key

Selection Sort

Selection-Sort Example

i	A	s
0	7, 4, 8, 2, 5, 3, 9	3
1	2, 4, 8, 7, 5, 3, 9	5
2	2, 3, 8, 7, 5, 4, 9	5
3	2, 3, 4, 7, 5, 8, 9	4
4	2, 3, 4, 5, 7, 8, 9	4
5	2, 3, 4, 5, 7, 8, 9	5
6	2, 3, 4, 5, 7, 8, 9	6

```
def selection_sort(A):
    n ← size(A)
    for i in [0:n] do
        # find s ≥ i minimizing A[s]
        s ← i
        for j in [i:n] do
            if A[j] < A[s] then
                s ← j
        # swap A[i] and A[s]
        A[i], A[s] ← A[s], A[i]
```

Sorted list Implementation

- **insert** in O(n) time since we have to find the correct place to insert
- **remove_min** and **min** in O(1) time since smallest value is always at the beginning

Insertion Sort

Insertion-Sort Example

i	A	j
1	7, 4, 8, 2, 5, 3, 9	0
2	4, 7, 8, 2, 5, 3, 9	2
3	4, 7, 8, 2, 5, 3, 9	0
4	2, 4, 7, 8, 5, 3, 9	2
5	2, 4, 5, 7, 8, 3, 9	1
6	2, 3, 4, 5, 7, 8, 9	6

```
def insertion_sort(A):
    n ← size(A)
    for i in [1:n] do
        x ← A[i]
        # move forward entries > x
        j ← i
        while j > 0 and x < A[j-1] do
            A[j] ← A[j-1]
            j ← j - 1
        # if j>0 ⇒ x ≥ A[j-1]
        # if j<i ⇒ x < A[j+1]
        A[j] ← x
```

Tree Based Priority Queues (Heaps)

A heap is a **binary tree** storing (key, value) items at its nodes, satisfying the following properties:

1. Heap-Order: for every node m ≠ root, key(m) ≥ key(parent(m))
2. Complete Binary Tree: let *h* be the height
   - every level i < h is full (i.e. there are 2^i nodes)
   - remaining nodes take leftmost positions of level h

Min Heap holds the minimum value at the root
Max Heap holds the maximum value at the root
Its height is at MOST log(n)

- Insertion: 1. Create a empty space after the "last node" for new node 2. Apply the heap-order property, i.e, if the node is smaller than its parent, swap
- Removal: 1. Swap the "last node" and the root 2. Apply the heap-order property

Heap-sort is the version of priority-queue sorting that implements the priority queue with a heap. It runs in **O(n log n)** time.

Method	Unsorted List	Sorted List	Heap
size, isEmpty	$O(1)$	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$	$O(\log n)$
min	$O(n)$	$O(1)$	$O(1)$
removeMin	$O(n)$	$O(1)$	$O(\log n)$
remove	$O(1)$	$O(1)$	$O(\log n)$
replaceKey	$O(1)$	$O(n)$	$O(\log n)$
replaceValue	$O(1)$	$O(1)$	$O(1)$

L6

Hashing

Map ADT

A way to represent key value pairs

Operations

- **get(k)**: if the map M has an entry with key k, return its associated value; else, return null
- **put(k, v)**: insert entry (k, v) into the map M; if key k is not already in M, then return null; else, return old value associated with k
- **remove(k)**: if the map M has an entry with key k, remove it from M and return its associated value; else, return null
- **size()**
- **is_empty()**
- **entries()**: return an iterable collection of the entries in M
- **keys()**: return an iterable collection of the keys in M
- **values()**: return an iterable collection of the values in M

Look Up Table

Consider a restricted setting in which a map has keys that are numbers, say in the range from 0 to N − 1

- Implement with an array of size N
- Key can be the index, so entries can be located directly
- O(1) operations (get, put, remove)

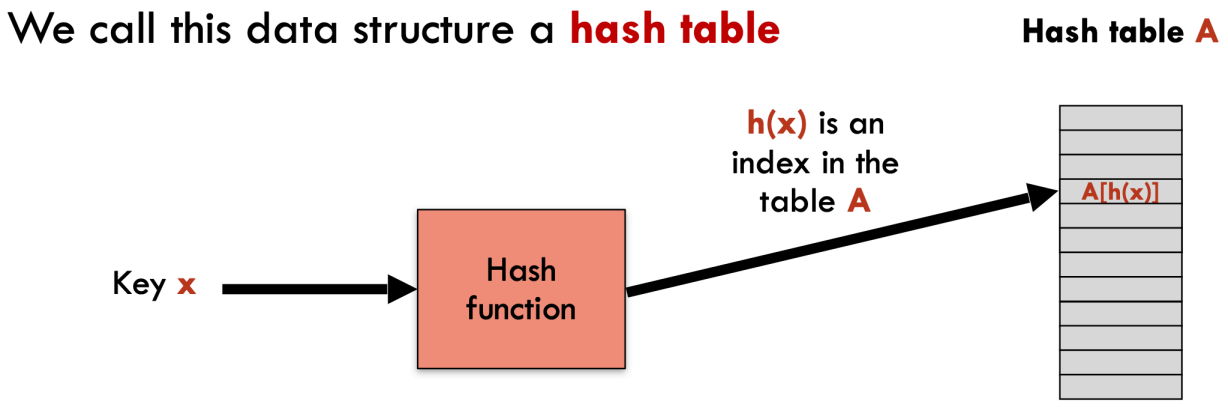
This give really good worst-case runtime, but bad space utilisation if we never use N keys

To save space

- List based map - get(): O(n)
  To save time
- lookup table - get(): O(1)

Hash Functions & Hash Table

We have a hash function h, which alwasy give the same answer for the same exact input, and is fairly efficient to computer - where h(x) is an input on a hash table.



Hash Function

- Function h: K → [0, N-1] that maps keys from a given set K of size m to integers in a fixed interval [0, N-1]
- Typically, N is much much smaller than m

Hash Value

- The integer h(x) is called the hash value of key x

Hash Table

- Consists of the hash function and the array (aka table) A of size N
- The cell A[h(x)] stores a pointer to the item (x, value)
- Each cell of A is sometimes called a bucket

Collisions

Two keys x, y collide if $h(x) = h(y)$

Universal Hash Functions

Suppose the key set is K = [0, M] and we need a hash table of size N. Let H be a family of such hash functions. We say that H is 2-universal if picking h uniformly at random (UAR) from H yields (for all i ≠ j):

$$Pr[h(i) = h(k)] \leq 1/N$$

Fact: Let h be a function chosen UAR from a 2-universal family H. Then the expected number of collisions for a given key with a set of n keys is at most n/N
In other words: If A has n keys, and k is a new key, we can expect k to collide with at most n/N of the keys already in the table

When collisions happen we have 3 methods for dealing with them

Seperate Chaining

Each cell of the hash table points to a linked list with any duplicate values in there. Simple, but requires additional memory outside the table.

Linear Probing

Place the colliding item into the next available cell. Colliding items lump together. Searching involves starting at correct cell, and moving until you find the item. We have to add a value **DEFUNCT** to differentiate between empty cells and deleted items.

Cuckoo Hashing

Uses two hash tables and two hash functions. This allows every value 2 places, and if they collide on one table, then one will go to its alternative location.
If it cant go to its alternative location, it evicts the current item and replaces it cell. The evicted item then goes to its alternative location, and if there is another item there, it evicts it and the cycle continues untill all items are settled. We can however get eviction cycles.

L7

Graphs

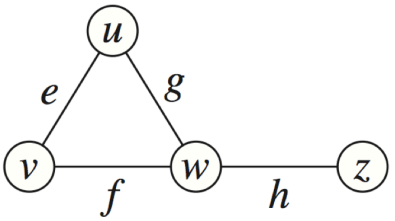
Edges, vertices, cycles, degrees, connectivity

Implementation

Edge List Structure

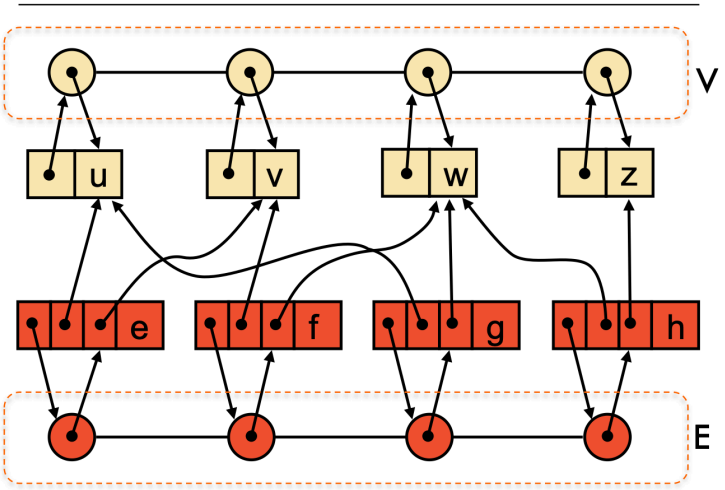
Vertex sequence holds

- sequence of vertices
- vertex object keeps track of its position in the sequence

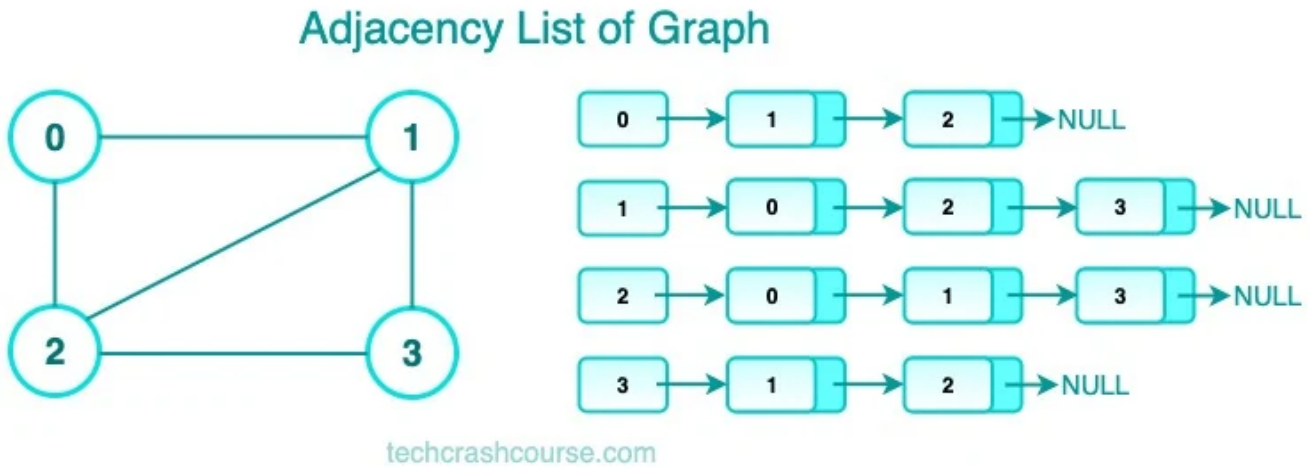


Edge sequence

- sequence edges
- edge object keeps track of its position in the sequence
- Edge object points to the two vertices it connects



Adjacency List

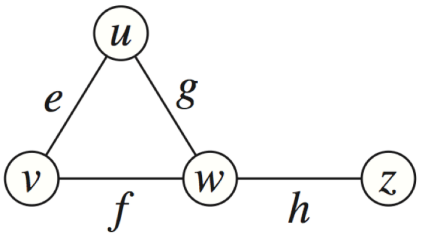


Adjacency Matrix Structure

Vertex array induces an index from 0 to n-1 for each vertex

2D-array adjacency matrix

- Reference to edge object for adjacent vertices
- Null for nonadjacent vertices

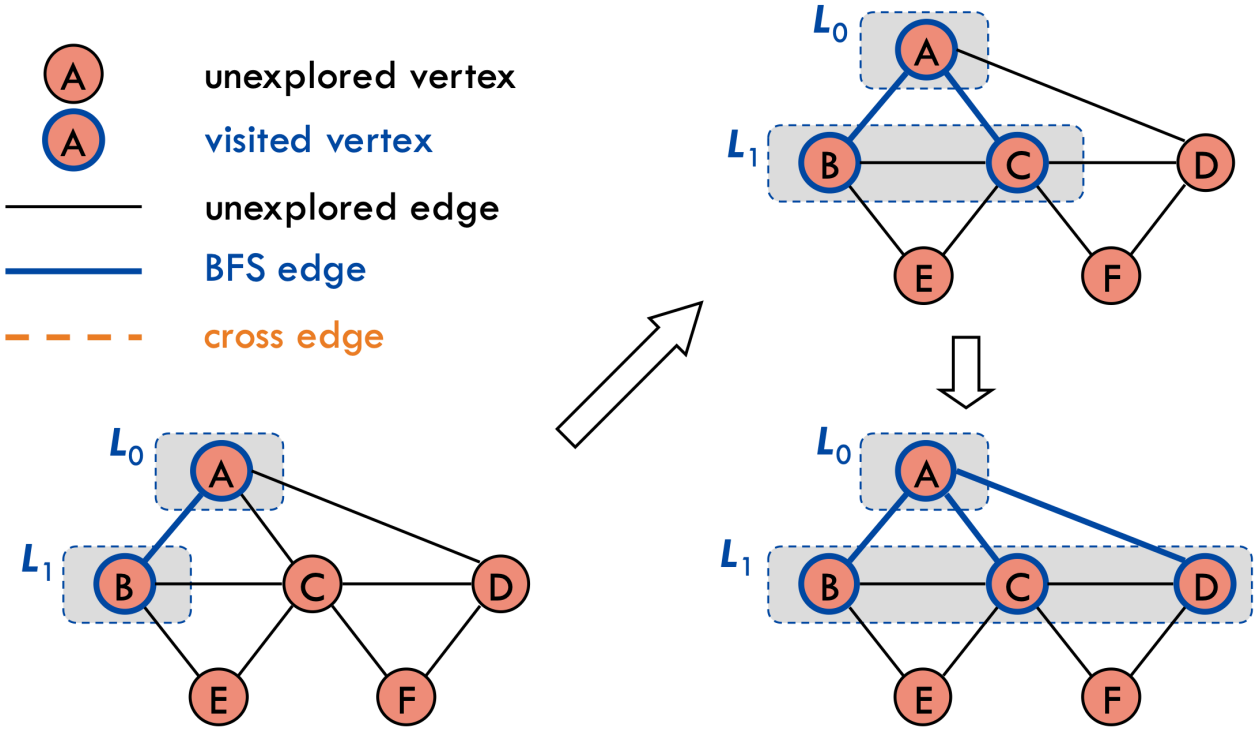


	0	1	2	3
u → 0		e	g	
v → 1	e		f	
w → 2	g	f		h
z → 3			h	

Graph Traversal

Breadth First Search (BFS)

This strategy tries to visit all vertices at distance k from a start vertex s before visiting vertices at distance k + 1:

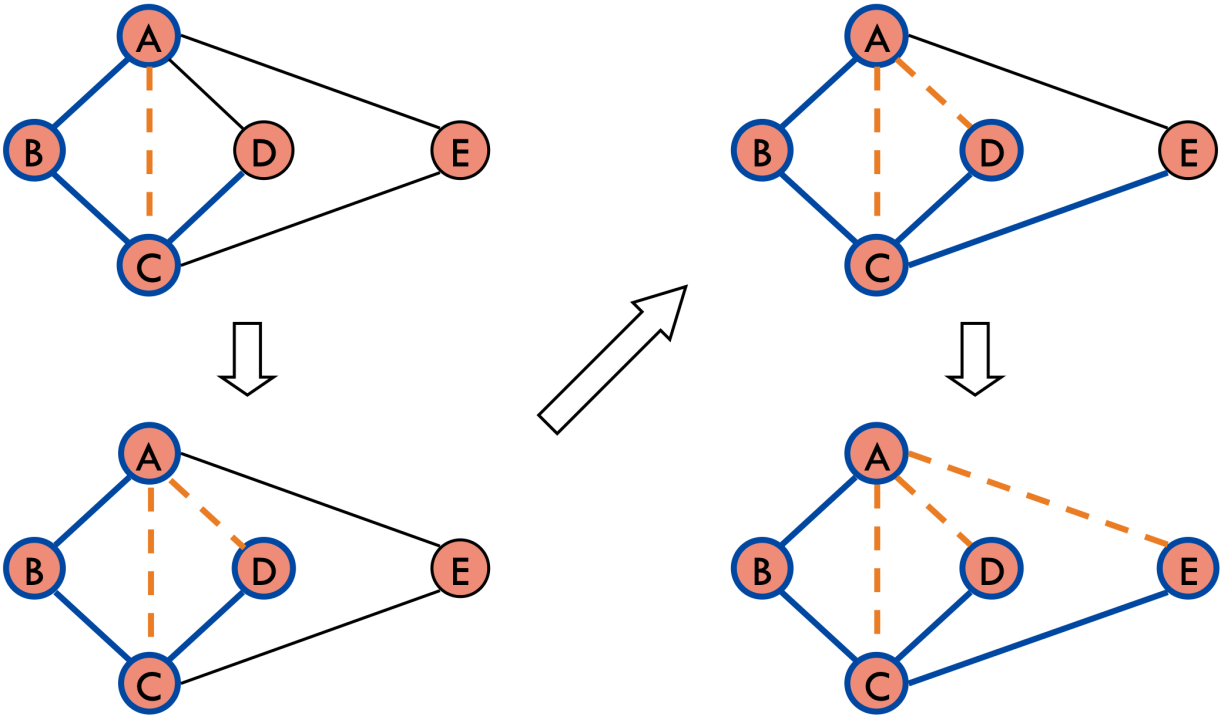


Assuming adjacency list representation we can perform a BFS traversal of a graph with n vertices and m edges in **O(n+m)** time

Assuming adjacency matrix representation we can perform a BFS traversal of a graph with n vertices and m edges in $O(n^2)$ time

Depth First Search (DFS)

This strategy tries to follow outgoing edges leading to yet unvisited vertices whenever possible, and backtrack if “stuck”. If an edge is used to discover a new vertex, we call it a DFS edge, otherwise we call it a back edge.
Assuming adjacency list represenation, DFS runs in **O(n+m)** time

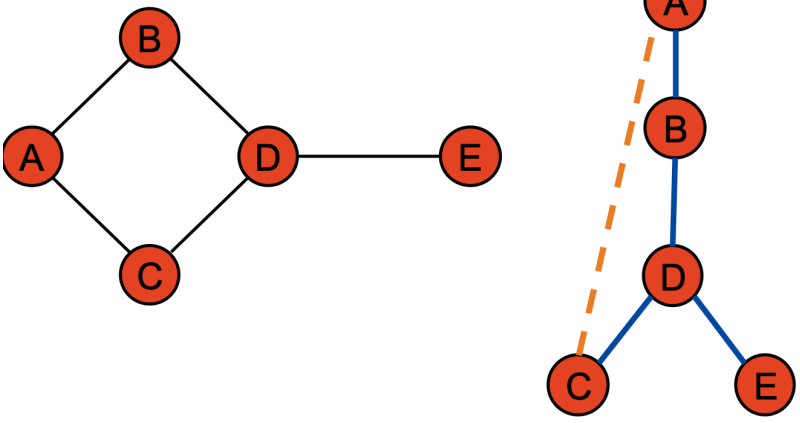


Identifying cut edges in O(n+m) time

Compute a DFS tree of the input graph G=(V, E)

For every u in V, compute level[u], its level in the DFS tree

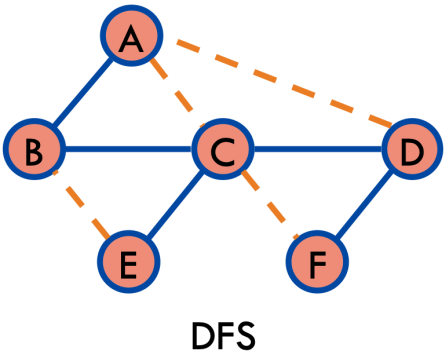
For every vertex v compute the shallowest level (closer to the root) that we can reach by taking DFS edges down the tree and then one back edge up. Call this down_and_up[v]



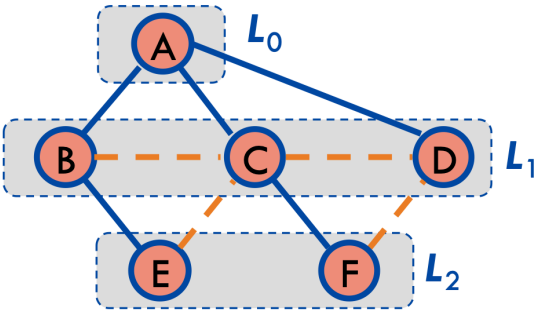
	level	d&u
A	0	0
B	1	0
C	3	0
D	2	0
E	3	3

BFS vs DFS

Applications	DFS	BFS
Spanning forest, connected components, paths, cycles	√	√
Shortest paths		√
Cut edges	√	



DFS



BFS

L8

Graphing Algorithms

Dijkstra's

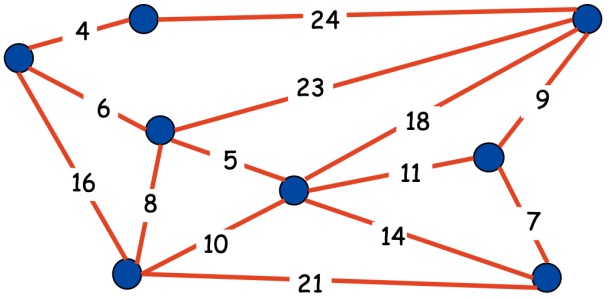
May not work for negative weight edges

```
1   def Dijkstra(G, w, s):
2       # initialize algorithm
3       for v in V do
4           D[v] ← ∞
5           parent[v] ← ∅
6       D[s] ← 0
7       Q ← new priority queue for { (v, D[v]) : v in V }
8
9       # iteratively add vertices to S
10      while Q is not empty do
11      u ← Q.remove_min()
12      for z in G.neighbors(u) do
13          if D[u] + w[u, z] < D[z] then
14              D[z] ← D[u] + w[u, z]
15              Q.update_priority(z, D[z])
16              parent[z] ← u
17      return D, parent
```

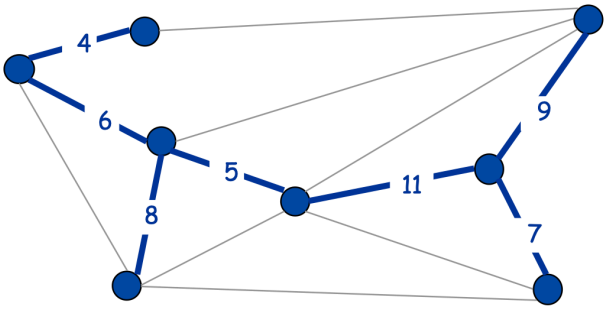
Using a heap for the Priority Queue (PQ), Dijkstra's algorithm runs in **O(m log n)** time. Fibonacci heap is a PQ that can carry out decrease key in O(1) amortized time. Using that instead we get **O(m + n log n)** time.

Minimum Spanning Tree (MST)

Given a connected graph G = (V, E) with real-valued edge weights c_e, a MST is a subset of the edges $T \subseteq E$ such that T is a spanning tree whose sum of edge weights is minimised.



G = (V, E)

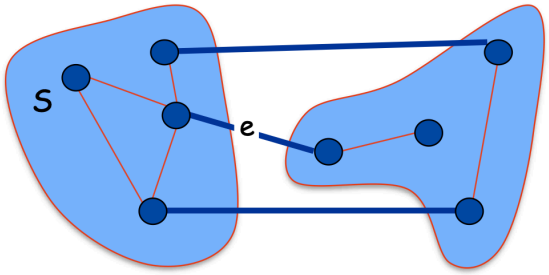


T with $\Sigma_{e \in T} c_e = 50$

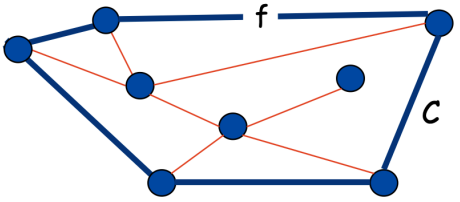
Properties

Cut Property: Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S. Then the MST contains e.

Cycle Property: Let C be any cycle, and let f be the max cost edge belonging to C. Then the MST does not contain f.

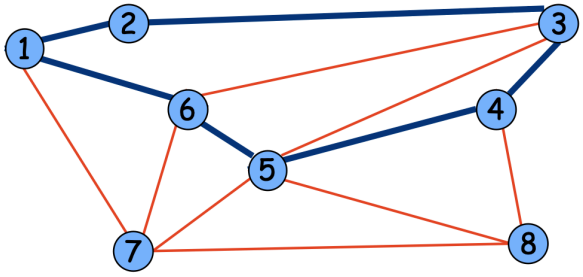


e is in the MST



f is not in the MST

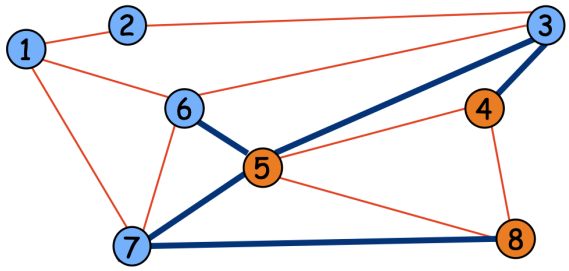
Cycles



Cycle C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1

Cutset

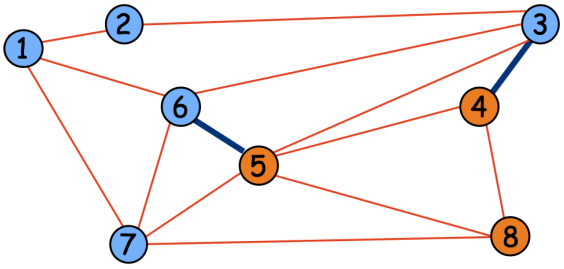
A cut is a subset of nodes S. The corresponding cutset D is the subset of edges with exactly one endpoint in S.



Cut S = { 4, 5, 8 }
Cutset D = 5-6, 5-7, 3-4, 3-5, 7-8

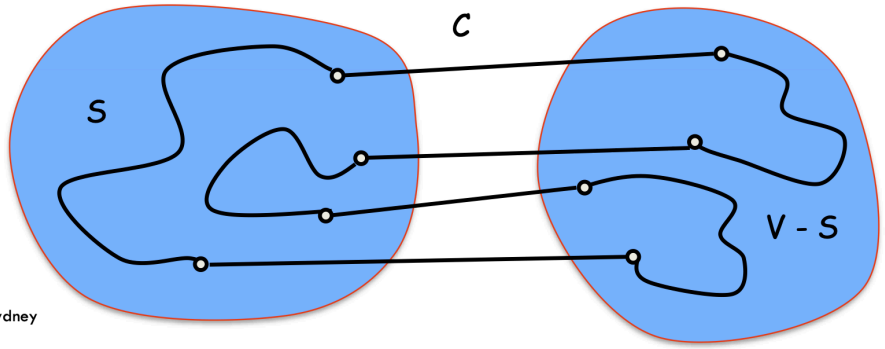
Cycle-Cut Intersection

Claim: A cycle and a cutset intersect in an even number of edges



Cycle C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1
Cutset D = 3-4, 3-5, 5-6, 5-7, 7-8
Intersection = 3-4, 5-6

Proof:



- See Prims Algorithm

Prims Algorithm

Finds the MST of a connected, weighted, undirected graph
Idea:

1. Pick any starting vertex.
2. Put it into a set S, where S means “vertices already included in the MST.”
3. Look at all edges leaving S, meaning edges with one endpoint inside S and one endpoint outside S.
4. Choose the cheapest such edge.
5. Add that edge to the MST.
6. Add the new vertex to S.
7. Repeat until every vertex is in S.

```
1   def prim(G, c):
```

```
    u ← arbitrary vertex in V
    S ← {u}
    T ← ∅

    while |S| < |V| do
        (u, v) ← min cost edge such that u in S and v not in S
        add (u, v) to T
        add v to S

return T
```

Always add the cheapest edge that connects the current tree to a vertex not yet in the tree.

Runs in **O(m log n)** time. When Fibonacci heap used it can be improved to **O(m + n log n)** time.

Kruskal's Algorithm

Finds the MST of a connected, weighted, undirected graph
Idea:

1. Sort all edges by increasing weight.
2. Start with an empty set of chosen edges T.
3. Look at the cheapest remaining edge.
4. If adding it creates a cycle, skip it.
5. If adding it does not create a cycle, add it to T.
6. Repeat until you have $n - 1$ edges, where n is the number of vertices.

```
def Kruskal(G, c):
    sort E in increasing c-value
    T ← ∅

    for each edge (u, v) in sorted E do
        if adding (u, v) does not create a cycle then
            add (u, v) to T

    return T
```

Always consider the cheapest unused edge, but only add it if it does not create a cycle.

Runs in **O(m log m)**. Also written as **O(m log n)** as in a simple graph m is about at most n^2.

L9

Greedy Algorithms

A greedy algorithm builds a solution by making the best local choice each step without considering past choices.
Greedy = locally optimal choice -> hope for global optimum.

It's not always correct!
Generic Structure:

1. Sort/order input
2. For each element
   - if it improves solution - take it
3. Return result

Fractional Knapsack

Given a set S of n items, each having:

- b_i : a positive benefit
- w_I : a positive weight

Goal: Choose items with maximum total benefit of weight at most W.
Solution: Pick item with highest value/weight ratio : $\frac{b_i}{w_i}$ to maximise value per unit weight.
Algorithm:

1. Compute value/weight for each item
2. Sort items in descending ratio
3. For each item:
   - take as much as possible
     Complexity:

- Sorting: O(n log n)
- Total: O(n log n)

An exchange argument proves a greedy algorithm is optimal by showing that any optimal solution can be transformed into the greedy solution without making it worse.

Interval Partitioning (Scheduling)

Problem: Schedule lectures using minimum classrooms. No two overlapping lectures in the same room. Depth = max number of overlapping intervals. Minimum rooms must be $\geq$ depth.
Greedy Strategy:

1. Sort lectures by start time
2. For each lecture:
   - assign to earliest available room
   - if non available --> open new room
     Complexity: O(n log n) - priority queue
     Greedy is optimal because it never "falls behind"

Text Compression - Huffman Encoding

- Let C be the set of characters in X
- Compute frequency f(c) for each character c in C
- Encode high-frequency characters with short code words
- No code word is a prefix of another code word
- Use an optimal encoding tree to determine the code words
  Problem: Compress text using fewer bits
  Suggestion:
- Frequent characters --> short codes
- Rare character --> long codes
- Use prefix codes (A prefix code is a code such that no code-word is the prefix of another code-word)
- NOTE - number of bits for a character = depth of character in tree
  Greedy Strategy:

1. Put all characters in a priority queue (by freq.)
2. Repeat:
   - remove 2 smallest
   - merge into new node
   - reinsert
3. Final tree = encoding

Essentially the most common items at the top of a binary tree, shortest prefix codes (0 for left, 1 for right), rare items near the bottom, longer prefix codes.
Complexity: O(d log d) where d = num of characters
Greedy works because optimal trees always combine lowest freq. nodes first.

```
1   def huffman(C, f):
2
3       # initialize priority queue
4       Q ← empty priority queue
5       for c in C do
6           T ← single-node binary tree storing c
7           Q.insert(f[c], T)
8
9       # merge trees while at least two trees
10      while Q.size() > 1 do
11          f1, T1 ← Q.remove_min()
12          f2, T2 ← Q.remove_min()
13          T ← new binary tree with T1/T2 as left/right subtrees
14          f ← f1 + f2
15          Q.insert(f, T)
16
17      # return last tree
18      f, T ← Q.remove_min()
19      return T
```

Greedy heuristics are easy to design but they are not always optimal. And when they are, small modifications of the problem can render them suboptimal:

- 0-1 knapsack is hard
- if tasks/lectures have special needs the problem is hard
- if we use non-binary encodings, Huffman does not work

L10/11

Divide and Conquer

Idea:

1. Divide
   If the input is small enough, solve it directly. Otherwise split the problem into smaller subproblems.
2. Recur/Delegate
   Recursively solve each subproblem. Assume the recursive calls correctly solve each subproblem
3. Conquer
   Combine the answers from the subproblems into the final solution

Examples

Binary search is the simplest divide and conquer algorithm example - already know it

Merge Sort

Given an unsorted array, return it in sorted order
Algorithm:

```
1   def merge_sort(S):
2       if |S| < 2:
3           return S
4
5       split S into left and right halves
6
7       sorted_left = merge_sort(left)
8       sorted_right = merge_sort(right)
9
10      return merge(sorted_left, sorted_right)
```

Divide, Recur, Conquer
Divide:

- Split array into 2 halfs
  Recur:

- Recusively sort each half
  Conquer:
- Merge the two sorted halves into one sorted array

Runtime: O(n log n) time

Quick Sort

Algorithm:

1. Choose a pivot.
2. Partition the array into:
   elements less than pivot
   elements equal to pivot
   elements greater than pivot
3. Recursively sort the less-than and greater-than lists.
4. Join:
   sorted less-than list
   equal-to-pivot list
   sorted greater-than list

Divide, Recure, Conquer:
Divide:
Pick a pivot and partition the array.
Recur:
Sort the less-than and greater-than partitions.
Conquer:
Concatenate the sorted pieces.

Runtime: Expected O(n log n) as random pivot

NOTE --> any comparison based sorting takes O(n log n) time

Maxima-Set / Pareto Frontier

Problem: Given a set of points in 2D, find all maximum points. A point is maximum is no other point is both to its right and above it. This is a Pareto Frontier
Algorithm:
Begins with preprocessing - sort points by increasing x-coordinates, break ties by increasing y-coordinate
Devide, Recure, Conquer:
Divide:
Split the sorted array into left and right halves.
Recur:
Find the maxima-set of each half.
Conquer:

Merge the two maxima-sets.
1. Find the highest point p in the right maxima-set.
2. For each point q in the left maxima-set:
if q.y > p.y:
keep q
otherwise:
discard q
3. Add every point from the right maxima-set.

Because the points are sorted, every point in the right half has larger x-coordinate than every point in the left half. Hence every point in the right maxima-set remains maximum in the whole set.
Runtime: O(n log n)

Integer Multiplication

Problem: Given two n-digit binary integers x and y, compute: $x \cdot y$
Solution:

```
def multiply(x, y):
# base case
⋮
# recursive case
    let x1 and x0 be such that x = x1 2n/2 + x0
    let y1 and y0 be such that y = y1 2n/2 + y0
    first_term ← multiply(x1, y1)
    last_term ← multiply(x0, y0)
    other_term ← multiply(x1 + x0, y1 + y0)

    return first_term 2n + (other_term − first_term − last_term) 2n/2 +
last_term
```

Runtime: Solves to $O(n^{\log_2 3})$, which is slightly better than the niave approach of $O(n^2)$

The general rule of solving runtime complexity with recurrences is: T(n) = recursive work + non-recursive work

Examples

Binary search:
T(n) = T(n/2) + O(1) = O(log n)

Merge sort:
T(n) = 2T(n/2) + O(n) = O(n log n)

Linked-list binary search:
T(n) = T(n/2) + O(n) = O(n)

Naïve divide-and-conquer multiplication:

$T(n) = 4T(n/2) + O(n) = O(n^2)$

Karatsuba multiplication:

$T(n) = 3T(n/2) + O(n) = O(n^{\log_2 3})$

Master Theorem

This is a theorem for recurrences in the form:

> $T(n) = aT(n/b) + f(n)$

where:

- a = number of recursive subproblems
- b = shrink factor
- f(n) = non-recursive work per level
  You compare f(n) to $n^{\log_b(a)}$ --> think of it like the recursive branching cost
  So the master theorem asks:
  Which dominates?

1. Recursive branching?
2. Work done at each level?
3. Are they balanced?

Step by step

1. Put the recurrence into the form: T(n) = aT(n/b) + f(n)
2. Identify a, b, and f(n).
3. Compute: n^(log_b a)
4. Compare f(n) with n^(log_b a).
5. Choose the matching case.

Comparison	Case	Result
`f(n)` smaller than critical	Case 1	`T(n) = θ(critical)`
`f(n)` same as critical	Case 2	`T(n) = θ(critical · log n)`
`f(n)` larger than critical	Case 3	`T(n) = θ(f(n))`

L12

Randomisation